Python for Geospatial – Part 1: Python Basics, GeoPandas & Vector Data

(30 MINUTES)

This learner note introduces the fundamentals of Python for geospatial analysis, focusing on the use of Python programming concepts and GeoPandas for managing and analysing vector spatial data. Participants will learn how Python supports modern GIS workflows through data structures, functions, spatial data handling, visualization, and core spatial analysis techniques such as buffering, spatial joins, and overlay operations. The module provides a practical foundation for applying Python in geospatial projects and prepares learners for more advanced topics in spatial data science and GeoAI.

Learning objectives

After completing this module, you will be able to:

- Explain the role of Python and key geospatial libraries in supporting spatial data analysis and GIS workflows.
- Load, explore, and visualize vector geospatial datasets using GeoPandas data structures.
- Apply fundamental spatial analysis techniques such as buffer analysis, spatial joins and overlay operations to solve common geospatial problems and derive spatial insights.

1. Introduction

Geospatial data is becoming increasingly important across many industries, including urban planning, environmental monitoring, transportation, agriculture, utilities, disaster management, and business intelligence. Modern GIS professionals are expected not only to use desktop GIS software but also to automate workflows and perform advanced spatial analysis. Python has emerged as one of the most widely used programming languages in the geospatial domain due to its simplicity, extensive ecosystem, and strong community support.

This session introduces the foundations of Python for geospatial analysis, focusing on:

- Python programming basics
- GeoPandas data structures
- Vector data management
- Spatial analysis techniques
- Spatial joins and overlays

By the end of this session, learners should understand how Python can be used to load, inspect, visualize, and analyse vector geospatial data.

2. Why Python for Geospatial Analysis?

Python provides a flexible and powerful platform for geospatial data processing. Unlike traditional GIS software that relies heavily on graphical interfaces, Python allows users to automate repetitive tasks and build reproducible workflows.

Benefits of Python:

- Automation
- Python can automate tasks such as:
- Data cleaning
- Coordinate conversion
- Batch processing
- Map generation
- Report production

Reproducibility

- Scripts can be reused and shared, ensuring consistent analysis across projects.

Integration

Python integrates easily with:

- GIS platforms
- Databases
- Web services
- Machine learning frameworks
- Cloud computing environments

3. Key Geospatial Libraries

GeoPandas

- GeoPandas extends the capabilities of Pandas by adding support for spatial data.

Common applications:

- Reading vector files
- Spatial queries
- Spatial joins
- CRS transformations
- Mapping

Rasterio

- Rasterio is used for raster data operations such as:
- Reading GeoTIFF files
- Accessing image bands
- Clipping rasters
- Raster calculations

Folium

Folium creates interactive web maps based on the Leaflet JavaScript library.

Applications include:

- Interactive dashboards
- Choropleth maps
- Marker clusters
- Spatial storytelling

Shapely

Shapely performs geometric operations including:

- Buffering
- Intersection
- Union
- Distance calculation

PyProj

PyProj manages coordinate reference systems and projections.

Applications include:

- Reprojection
- Coordinate transformation
- CRS validation

4. Fundamental Python Data Structures

Before working with spatial data, it is essential to understand Python's basic data structures.

Lists

Lists are ordered collections that can store multiple items.

Example:

```
cities = ["Kuala Lumpur", "Johor Bahru", "Penang"]
```

Geospatial applications:

- Coordinate storage
- Route sequences
- Feature collections

Example:
```
coordinates = [
    (3.1390,101.6869),
    (1.4927,103.7414)
]
```

Characteristics:

- Ordered
- Mutable
- Allow duplicates

Dictionaries

Dictionaries store information as key-value pairs.

Example:
```
city = {
    "name": "Kuala Lumpur",
    "latitude": 3.1390,
    "longitude": 101.6869
}
```

Geospatial applications:

- Feature attributes
- Metadata
- Configuration settings

Characteristics:

- Fast lookup
- Flexible structure
- Human-readable

5. Functions and Python Packages

Functions

Functions allow code to be reused and organised into manageable components.
Example:
```
def calculate_buffer(gdf, distance):
    return gdf.buffer(distance)
```

Advantages:

- Reusability

- Reduced duplication
- Improved readability
- Easier maintenance

Importing Packages

- Python libraries must be imported before use.

Example:

```
import geopandas as gpd
import pandas as pd
from shapely.geometry import Point
```

Common conventions:

Library	Alias
Pandas	pd
GeoPandas	gpd
NumPy	np
Matplotlib	plt

6. Understanding Vector Data

Vector data represents geographic features using coordinates.

There are three primary geometry types:

Point - Represents a single location.

Examples:

- Schools
- Hospitals
- Weather stations
- Trees

Line - Represents linear features.

Examples:

- Roads
- Rivers
- Utility networks
- Pipelines

Polygon - Represents areas.

Examples:

- Administrative boundaries
- Land parcels
- Forest reserves
- Lakes

Vector Data Components

Each vector dataset contains:

- Geometry - the spatial representation.
- Attributes - Descriptive information associated with each feature.

Example:

School_ID	Name	District
101	ABC School	Petaling
102	XYZ School	Klang

7. GeoSeries and GeoDataFrame

GeoPandas introduces two important spatial data structures.

GeoSeries - a collection of geometry objects.

Examples:

- Points
- Lines
- Polygons

Example:

- gdf.geometry

Common operations:

- gdf.geometry.area
  gdf.geometry.length
  gdf.geometry.centroid

GeoDataFrame - extends a Pandas DataFrame by including a geometry column.

Example:

- `gdf = gpd.read_file("schools.shp")`

Structure:

Name	Type	Geometry
School A	Primary	POINT
School B	Secondary	POINT

Benefits:

- Spatial analysis
- Mapping
- CRS management
- Spatial indexing

Geometry Column

- A GeoDataFrame may contain multiple geometry columns but only one active geometry column.

Example:

- `gdf.set_geometry("new_geom")`

Coordinate Reference Systems (CRS) - A CRS defines how coordinates relate to locations on Earth.

Example:

- `gdf.crs`

Common CRS:

CRS	EPSG
WGS84	4326
Web Mercator	3857
UTM Zones	Various

Reprojection:

- `gdf = gdf.to_crs("EPSG:3857")`

8. Reading and Exploring Vector Data

GeoPandas supports many vector formats.

***Shapefile* -** Traditional GIS format.

Components:

- .shp
- .dbf
- .prj
- .shx

Example:

- `gdf = gpd.read_file("roads.shp")`

GeoJSON - JSON-based spatial format.

Advantages:

- Lightweight
- Web-friendly
- Human-readable

Example:

- `gdf = gpd.read_file("data.geojson")`

GeoPackage - Modern OGC standard.

Advantages:

- Single file
- Multiple layers
- Large dataset support

Example:

- ```
  gdf = gpd.read_file(
      "data.gpkg",
      layer="boundaries"
  )
  ```

Data Exploration

Common commands:

```
gdf.head()
gdf.info()
gdf.describe()
gdf.columns
```

These help users understand:

- Data structure
- Attributes
- Missing values
- Geometry types

9. Visualising Vector Data

GeoPandas provides built-in plotting capabilities.

Basic plot:

- `gdf.plot()`

Adding title:

```
gdf.plot()
plt.title("Road Network")
```
```

Choropleth Mapping

- A choropleth map visualises attribute values using colour gradients.

Example:

```
gdf.plot(
    column="population",
    legend=True
)
```

Applications:

- Population density
- Income distribution
- Land use analysis
- Environmental indicators

10. Spatial Analysis Operations

Spatial analysis is the core strength of GIS.

Buffer Analysis

- Buffers create zones around features.

Example:

- `schools.buffer(500)` // Creates a 500-meter buffer around schools.

Applications:

- Accessibility analysis
- Service coverage
- Impact assessment
- Environmental monitoring

Spatial Join

- Spatial joins transfer attributes between layers based on location.

Example:

```
gpd.sjoin(
    schools,
    districts,
    predicate="within"
)
```

Applications:

- Assign schools to districts
- Link properties to zones
- Associate facilities with administrative areas

Nearest Join

- Matches each feature with its closest neighbouring feature.

Example:

- `gpd.sjoin_nearest()`

Applications:

- Nearest hospital
- Nearest bus stop
- Nearest emergency facility

Overlay Analysis

- Overlay combines multiple layers to evaluate spatial relationships.

Common operations:

- Intersection
- Keeps overlapping areas only.
- Union
- Combines all areas.
- Difference
- Removes overlapping portions.

Example:

```
gpd.overlay(
    flood_zone,
    parcels,
    how="intersection"
)
```

Applications:

- Flood-risk assessment
- Environmental impact studies
- Land suitability analysis
- Urban planning

11. Summary and Key Takeaways

Python is a powerful platform for modern geospatial analysis.

Key concepts covered include:

- Python fundamentals
- Lists and dictionaries
- Functions and packages
- Vector data structures
- GeoSeries and GeoDataFrames

- Coordinate Reference Systems
- Reading spatial data
- Vector visualisation
- Buffer analysis
- Spatial joins
- Overlay operations

These concepts form the foundation for more advanced geospatial workflows involving raster analysis, coordinate transformations, machine learning, GeoAI, and spatial data science.